

Predicting Transport Cost for Medical Equipment Deliveries

Machine Learning Project-Checkpoint 1

Team: Unsupervised Learners

Sawant Hrushikesh Rahul (IMT2023619), Akshat Mittal (IMT2023606)

October 26, 2025

Contents

1	Problem Statement and Data Description	3
1.1	Background	3
1.2	Dataset Overview	3
1.3	Columns / Feature Descriptions	3
1.4	Problem Type and Evaluation Metric	4
1.5	Libraries and Tools Used	4
2	Models, Preprocessing and Results	5
2.1	Linear Regression	5
2.2	Random Forest Regressor	6
2.3	XGBoost Regressor	9
2.4	AdaBoost Regressor	10
2.5	CatBoost Regressor	12
2.6	Gradient Boosting Regressor	14
3	Comparisons of models on the basis of their respective kaggle score	17
4	Hyperparameter tuning analysis and optimization history plot	18
5	GitHub repository link	19

1 Problem Statement and Data Description

1.1 Background

Transporting sensitive medical equipment presents unique logistical and operational challenges, including:

- Ensuring fragile and high-value devices arrive undamaged.
- Handling urgent shipments for critical hospital needs with minimal delay.
- Managing cross-border deliveries and addressing rural/remote hospital locations.
- Coordinating additional services such as on-site installation and setup.

Our company sources medical equipment from multiple suppliers worldwide and provides delivery services to hospitals. The objective of this project is to **build a supervised machine learning model to predict the transport cost** of each order. Reliable predictions will enable better route planning, resource allocation, dynamic pricing, and improved customer communication.

1.2 Dataset Overview

The dataset provided contains the following files:

- `train.csv`: 5000 samples, 20 columns (includes `Transport_Cost` target).
- `test.csv`: 500 samples, 19 columns (target excluded).
- `sample_submission.csv`: template for submission.

[Click here to navigate to the Kaggle Competition](#)

1.3 Columns / Feature Descriptions

Table 1 lists the features available in the dataset and short descriptions for each.

Table 1: Dataset features and descriptions

Column Name	Description
Hospital_Id	Unique identifier for the hospital.
Supplier_Name	Name of the equipment supplier.
Supplier_Reliability	Supplier reputation score (higher = more reliable).
Equipment_Height	Height of the equipment (units as provided).
Equipment_Width	Width of the equipment.
Equipment_Weight	Weight of the equipment.
Equipment_Type	Categorical descriptor of equipment (e.g., diagnostic, surgical).
Equipment_Value	Monetary value of the equipment (currency units).
Base_Transport_Fee	Base transport fee charged before extras.
CrossBorder_Shipping	Boolean/category indicating international delivery.
Urgent_Shipping	Whether shipment was express/urgent.
Installation_Service	Whether installation/setup service was requested/included.
Transport_Method	Mode of transport (air, road, sea, etc.).
Fragile_Equipment	Whether the equipment is labelled fragile.
Hospital_Info	Additional textual metadata about the hospital.
Rural_Hospital	Boolean indicating if hospital is located in a rural/remote area.
Order_Placed_Date	Date when order was placed.
Delivery_Date	Date when delivery occurred.
Hospital_Location	Geographical description (city / region / country).
Transport_Cost	Target variable: cost of transport for the order.

1.4 Problem Type and Evaluation Metric

This is a supervised **regression** problem where the target is **Transport_Cost**. Typical evaluation metrics include:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)

1.5 Libraries and Tools Used

The analysis and model development were carried out in **Python 3.10** using several open-source machine learning and data analysis libraries. The key libraries utilized are:

- **pandas** - for data loading, cleaning, preprocessing, and feature engineering.
- **NumPy** - for numerical operations, vectorized computations, and efficient array handling.
- **scikit-learn** - for machine learning algorithms, model evaluation (e.g., K-Fold Cross-Validation, RMSE), preprocessing utilities, and ensemble models such as Random Forest, Gradient Boosting, and AdaBoost.
- **Linear regression** - done for both lasso and ridge regularization

- **XGBoost** - for gradient-boosted decision trees optimized for performance and scalability.
- **CatBoost** - for categorical boosting with built-in handling of categorical features.
- **Optuna** - for hyperparameter optimization using the Tree-structured Parzen Estimator (TPE) sampler and visualization of optimization history.
- **Matplotlib** - for visualizing optimization progress and model performance curves.

All experiments were executed in a controlled Jupyter Notebook environment to ensure reproducibility, with random seeds fixed at 42 wherever applicable.

2 Models, Preprocessing and Results

2.1 Linear Regression

Model overview. As a baseline, standard regularized linear models (**Ridge** and **Lasso** from `scikit-learn`) were also trained. These models are highly interpretable and fast to train. The pipeline required one additional critical step not used by the tree-based models: feature scaling. Linear models are sensitive to the scale of input features, so standardization was applied after all other preprocessing. The models were trained on the log $1p$ -transformed target.

Preprocessing pipeline (explicit steps and ordering). The preprocessing steps are nearly identical to those in Section 2.6, with the crucial addition of feature scaling.

1. **Numeric missing-value imputation.** Fill missing numeric features with their respective medians computed on the training set.
2. **Categorical missing-value imputation.** Fill missing categorical features with their respective modes computed on the training set.
3. **Date parsing and date-derived features.** Parse date columns and derive features like `Delivery_Time_Days`, `Order_Year`, `Order_Month`, and `Order_Day_of_Week`.
4. **Physical / value feature engineering.** Compute domain-informed features like `Value_per_Weight`, `Equipment_Area`, and `Is_Weekend`.
5. **Binary mapping for boolean-like columns.** Convert yes/no fields to numeric flags (1/0) and fill missing with 0.
6. **Hospital_Info mapping.** Map `Hospital_Info` to numeric values (`{Wealthy:1, WorkingClass:0}`).
7. **One-hot encoding for multi-class categoricals.** Apply `pd.get_dummies(..., drop_first=True)` to `Equipment_Type` and `Transport_Method`.
8. **Feature Scaling.** After all feature engineering and encoding, the resulting feature set is scaled using `StandardScaler`. This step is essential for regularized models like Ridge and Lasso to ensure all features contribute fairly to the regularization penalty.
9. **Drop identifiers and raw date columns.** Remove non-predictive columns like `Hospital_Id`, `Supplier_Name`, etc..
10. **Target transform.** The `Transport_Cost` target is transformed using log $1p$ before training.

Hyperparameter tuning and Validation. Hyperparameter tuning for Ridge and Lasso involves finding the optimal regularization strength (the `alpha` parameter). This was not done with Optuna. Instead, the standard approach is to use k-fold cross-validation (e.g., 5-fold `KFold`) combined with a `GridSearchCV` over a logarithmic range of `alpha` values. The model with the `alpha` that minimizes the mean cross-validated RMSE (computed in original cost units) would be selected for final training.

2.2 Random Forest Regressor

Model overview. A Random Forest regression model (scikit-learn’s `RandomForestRegressor`) is trained to predict `Transport_Cost`. The pipeline applies a sequence of deterministic pre-processing and feature engineering steps, uses a log-transform on the target, and performs fine-grained hyperparameter tuning with Optuna using a 5-fold `KFold` cross-validation. Final predictions are produced by re-training the best model on the entire training set.

Preprocessing pipeline (exact steps and ordering). Below we describe the preprocessing applied in the code in the exact order it is executed and why each step is used.

1. **Missingness indicator for supplier reliability.**

`Supplier_Reliability_is_Missing` is created as a binary flag (1 if missing, 0 otherwise). This preserves information about missingness which can be predictive (e.g., unknown reliability might correlate with higher transport costs).

2. **Train-time statistics / fills.**

When `is_train=True` the function computes a small dictionary of fill/statistics:

- `Supplier_Reliability_fill = -1` (explicit sentinel for missing reliability).
- Medians for numeric geometry/value features: `Equipment_Height_median`, `Equipment_Width_median`, `Equipment_Weight_median`, `Equipment_Value_median`.

At inference time (`is_train=False`) these saved stats are reused to guarantee identical imputation.

3. **Numeric imputation.**

Numeric columns `Equipment_Height`, `Equipment_Width`, `Equipment_Weight`, `Equipment_Value` are filled with the corresponding medians from training. `Supplier_Reliability` missing values are filled with `-1` (the sentinel).

4. **Categorical missing fill (simple).**

`Equipment_Type`, `Transport_Method`, and `Rural_Hospital` are filled with the string `'Unknown'` if missing. This preserves records and creates an explicit category for missing.

5. **Datetime parsing & derived date features.**

`Order_Placed_Date` and `Delivery_Date` are parsed with `pd.to_datetime(..., errors='coerce')`. Derived features:

- `Delivery_Time_Days = (Delivery_Date - Order_Placed_Date).days`, negative or NaN values are clipped to 0.
- `Order_Year`, `Order_Month`, `Order_Day_of_Week`, `Order_Day_of_Year`, `Order_Quarter`.
- `is_month_start = 1` if order day is 1 or 2 (encoded as small-month-start indicator).
- `is_month_end` flagged using `dt.is_month_end`.

Any NaNs after extraction are filled using the mode of that field computed on training (stored in `train_stats`).

6. Physical / value feature engineering.

The pipeline creates interaction / derived numeric features to better capture geometry and value relationships:

- $\text{Equipment_Area} = \text{Equipment_Height} \times \text{Equipment_Width}$.
- $\text{Value_per_Weight} = \frac{\text{Equipment_Value}}{\text{Equipment_Weight} + 1 \times 10^{-6}}$.
- $\text{Equipment_Density} = \frac{\text{Equipment_Weight}}{\text{Equipment_Area} + 1 \times 10^{-6}}$.
- $\text{Value_per_Area} = \frac{\text{Equipment_Value}}{\text{Equipment_Area} + 1 \times 10^{-6}}$.

Small epsilons (1e-6) avoid division-by-zero errors.

7. Binary encoding for yes/no columns.

The following columns are mapped $\{\text{'Yes':1, 'No':0}\}$ and missing $\rightarrow 0$: `CrossBorder_Shipping`, `Urgent_Shipping`, `Installation_Service`, `Fragile_Equipment`, `Rural_Hospital`.

Note: `Hospital_Info` is explicitly mapped $\{\text{'Wealthy':1, 'Working Class':0}\}$ then filled with 0 (this is a bespoke mapping in the code).

8. One-hot encoding for multi-class categoricals.

`pd.get_dummies(..., drop_first=True)` is applied to `Equipment_Type` and `Transport_Method`. Dropping the first column avoids collinearity and reduces dimensionality.

9. Interaction features.

Three multiplicative interactions are added: `Urgent_x_CrossBorder`, `Fragile_x_Rural`, `Fragile_x_Urgent`. These capture combined effects (e.g., urgent international fragile shipment).

10. Dropping identifiers and raw dates.

The following columns are dropped as they are identifiers or raw date fields: `Hospital_Id`, `Supplier_Name`, `Hospital_Location`, `Order_Placed_Date`, `Delivery_Date`. This avoids leaking raw identifiers and reduces redundancy.

11. Target transform (applied outside the preprocess function).

`Transport_Cost` is clipped at 0 and then transformed by $\log 1p$ (i.e., $\log(1+y)$). The model is trained to predict $\log 1p(\text{Transport_Cost})$. At inference time predictions are inverted with `np.exp` to get back dollar (original) units.

Why this pipeline (brief rationale).

- Imputation with medians keeps the numeric distributions robust to outliers and preserves scale for tree-based models. The -1 sentinel for supplier reliability allows the model to learn from “unknown” reliability explicitly.
- Date-derived features and delivery lead time capture temporal patterns (seasonality, weekday effects, lead time impact).
- Geometric/value-derived features (area, density, value-per-weight) capture physics/price relationships that strongly influence transport pricing.
- One-hot encoding for categorical variables and binary mapping for Yes/No columns are natural choices for tree ensembles (they handle sparse dummies and binary flags well).

- Log-transforming the target stabilises variance and reduces the influence of extreme transport costs during training; evaluating RMSE in original dollar units is performed by inverse-transforming predictions prior to computing RMSE.

Hyperparameter tuning (exact procedure and settings).

- **Tuning framework:** Optuna with the TPE sampler (`optuna.samplers.TPESampler(seed=42)`). The study is created with `direction='minimize'` to minimize validation RMSE.
- **Objective function:** For each trial a model (Random Forest or XGBoost depending on subsection) is constructed with parameters sampled by Optuna. The objective computes the mean RMSE over a 5-fold KFold (shuffled, `random_state=42`). For XGBoost specifically:
 1. Inside each fold the training portion is further split into a small training and an evaluation subset (15% of fold train) to enable *early stopping*.
 2. The model is trained on the inner training split with early stopping monitored on the inner eval split.
 3. Predictions on the K-Fold validation set are inverse-transformed from $\log 1p$ and RMSE is computed in original cost units.
- **Cross-validation:** 5-fold KFold with shuffle and fixed seed.
- **Search space (XGBoost as implemented):**
 - `n_estimators`: integer in [1000, 2500]
 - `learning_rate`: float in [0.05, 0.15] (log sampling)
 - `max_depth`: integer in [2, 5]
 - `subsample`: float in [0.75, 0.95]
 - `colsample_bytree`: float in [0.65, 0.85]
 - `gamma`: float in [0.01, 0.1] (log sampling)
 - `reg_alpha`: float in [1e-6, 1e-3] (log sampling)
 - `reg_lambda`: float in [0.01, 0.1] (log sampling)
 - `tree_method`: fixed to `hist`; `early_stopping_rounds`: 50 (used inside fold)
- **Sampling / trials:** `study.optimize(objective, n_trials=300)` — the code runs 300 trials of TPE sampling.
- **Metric used for optimization:** mean validation RMSE (in original cost units; computed after inverting the log transform).
- **Final selection and retraining:** After the study completes, `study.best_params` are retrieved and small keys not valid for final constructor (like `early_stopping_rounds`) are popped. The final XGBoost model is re-instantiated with `XGBRegressor(**best_params)` and trained on the full training set (features $\rightarrow$ $\log 1p$ target). Final predictions are inverse-transformed and clipped to be non-negative.

2.3 XGBoost Regressor

Model overview. An XGBoost regression model (`xgboost.XGBRegressor`) is trained using the same preprocessing pipeline as the Random Forest variant. This pipeline uses a log-transformed target, rich feature engineering, and Optuna-based hyperparameter tuning. For XGBoost the tuning loop incorporates inner-fold early stopping (by holding out a small eval set inside each training fold) to control overfitting and to allow very large `n_estimators` while avoiding wasted iterations.

Preprocessing pipeline (exact steps and ordering). The preprocessing steps are identical to those described in Section 2.2 and are applied before training the XGBoost model. In short:

1. Create `Supplier_Reliability_is_Missing` flag and compute train medians / fills.
2. Impute numeric columns with medians and fill categorical missing values with `'Unknown'`.
3. Parse dates, derive `Delivery_Time_Days` and many order-related features, and fill modes for date-derived NaNs.
4. Engineer physical/value features: `Equipment_Area`, `Value_per_Weight`, `Equipment_Density`, `Value_per_Area`.
5. Binary-map yes/no columns and custom-map `Hospital_Info`.
6. One-hot encode `Equipment_Type` and `Transport_Method` with `drop_first=True`.
7. Add interaction features (`Urgent_x_CrossBorder`, `Fragile_x_Rural`, `Fragile_x_Urgent`).
8. Drop identifiers and raw date columns.
9. Transform target with $\log 1p$ before training.

Hyperparameter tuning for XGBoost (exact procedure).

- **Sampler / study:** Optuna TPE sampler with seed 42; `direction='minimize'` on RMSE.
- **Per-trial evaluation:** For each trial XGBoost parameters are sampled from the focused search space. A 5-fold KFold with shuffle is used. Inside each fold:
 - The fold’s training portion is split with `train_test_split(..., test_size=0.15)` to form an inner evaluation set for early stopping.
 - `model.fit(..., eval_set=[(X_eval_fold, y_eval_fold)], verbose=False)` is called so the model uses early stopping (50 rounds) and selects the best iteration for that fold.
 - Predictions for the fold validation set use the best iteration (if available) and are inverse-transformed via `np.expm1` before RMSE is computed against original-dollar targets.
- **Search space:** See the list in the previous hyperparameter section (wide `n_estimators` range, learning rate around 0.05–0.15, shallow `max_depth` 2–5, and regularisation / subsample / column subsample sampled).
- **Trials and stopping:** `n_trials=300` with TPE sampling. Early stopping inside folds prevents wasteful training for large `n_estimators`.
- **Metric:** Mean 5-fold RMSE in original cost units (after inverting log transform).

- **Final model:** After finding `best_params`, keys not applicable to final constructor are removed (e.g., `early_stopping_rounds` if present). The final `XGBRegressor(**best_params)` is trained on full data (log target) and used to predict test set values; predictions are transformed back with `np.exp` and clipped to ≥ 0 .

Notes and best practices.

- Inner-fold early stopping is a robust way to allow very large `n_estimators` ranges while preventing overfitting; it also gives a per-fold best-iteration estimate that can be used when predicting the held-out fold.
- Using log-target training but computing RMSE in original-dollar units means the sampler optimizes a business-relevant metric (RMSE in dollars) even though the model trains on a stabilized target.
- If the data has chronological structure consider replacing shuffled `KFold` with a time-aware split to avoid leakage.

2.4 AdaBoost Regressor

Model overview. An AdaBoost regressor (`sklearn.ensemble.AdaBoostRegressor`) is trained to predict `Transport_Cost`. The pipeline uses deterministic, hand-crafted preprocessing and feature engineering, trains the model on a log-transformed target, and performs hyperparameter optimization with Optuna. The final model is re-trained on the full training set with the best hyperparameters and used to generate the submission file `submission_ada_hyp_300.csv`.

Preprocessing pipeline (explicit steps and ordering). All preprocessing below is performed before model training and is applied consistently at inference time using the same stored statistics.

1. Numeric missing-value imputation.

Compute training medians for the following numeric features and use them to fill missing values:

- `Supplier_Reliability_median`
- `Equipment_Height_median`
- `Equipment_Width_median`
- `Equipment_Weight_median`
- `Equipment_Value_median`

At inference, the saved medians are reused to ensure identical imputation.

2. Categorical missing-value imputation.

Compute the mode (most frequent value) on training and use it to fill missing entries for: `Equipment_Type`, `Transport_Method`, and `Rural_Hospital`.

3. Datetime parsing and date-derived features.

Parse `Order_Placed_Date` and `Delivery_Date` with `pd.to_datetime(..., errors='coerce')`. Create:

- `Delivery_Time_Days` = days between delivery and order (negative or missing values clipped to 0).
- `Order_Year`, `Order_Month`, `Order_Day_of_Week`.

Any missing date-derived values are filled with the corresponding mode computed on the training set.

4. Additional feature engineering.

Create physically and temporally meaningful features:

- $\text{Value_per_Weight} = \frac{\text{Equipment_Value}}{\text{Equipment_Weight} + 1}$
- $\text{Equipment_Area} = \text{Equipment_Height} \times \text{Equipment_Width}$
- $\text{Is_Weekend} = 1$ if Order_Day_of_Week is Saturday/Sunday (5,6), else 0

5. Binary mapping for yes/no columns.

Convert the following to numeric flags (1/0). The code handles presence/absence of the Yes label robustly and fills missing with 0: `CrossBorder_Shipping`, `Urgent_Shipping`, `Installation_Service`, `Fragile_Equipment`, `Rural_Hospital`.

6. Hospital_Info mapping.

Map `Hospital_Info` to numeric: `{Wealthy:1, Working Class:0}`. If `Wealthy` does not appear in the column, the field is mapped to 0 and missing values are filled accordingly.

7. One-hot encoding for multi-class categoricals.

Apply `pd.get_dummies(..., drop_first=True)` to `Equipment_Type` and `Transport_Method` to produce dummy variables while avoiding perfect collinearity.

8. Drop identifiers and raw dates.

Remove the following non-predictive or potentially leaking columns: `Hospital_Id`, `Supplier_Name`, `Hospital_Location`, `Order_Placed_Date`, `Delivery_Date`.

9. Target handling.

`Transport_Cost` is clipped at 0 and then transformed with $\log 1p$ (i.e., $\log(1 + y)$). The model is trained to predict the log-transformed target; predictions are inverse-transformed with $\exp(\cdot) - 1$ to report RMSE and to create submission values.

Hyperparameter tuning (procedure and settings). Hyperparameter optimization is performed using Optuna:

- **Optimization library:** Optuna (default sampler), minimizing validation RMSE.
- **Data split used for tuning:** A single static 80/20 split using `train_test_split(..., test_size=0.2, random_state=42)`:
 - `X_train_split`, `y_train_split` (80%) used for training inside each trial.
 - `X_val`, `y_val` (20%) used as the hold-out validation set for trial evaluation.
- **Target during tuning:** The training split target is transformed with $\log 1p$ (i.e., `y_train_split_log = log1p(y_train_split)`). Models are trained to predict the log-target; predictions on the validation set are inverse-transformed for RMSE computation.
- **Search space:**
 - `n_estimators`: integer in `[50, 500]`
 - `learning_rate`: float in `[0.001, 1.0]` (log-uniform sampling)
 - `loss`: categorical in `{linear, square, exponential}`
 - `random_state`: fixed to 42 (for reproducibility)
- **Trials:** `n_trials = 300` Optuna trials are executed.

- **Evaluation metric:** Root Mean Squared Error (RMSE) computed in original cost units after applying $\exp(\cdot) - 1$ to model predictions.
- **Final selection and retraining:** After optimization, `study.best_params` are retrieved and used to instantiate an `AdaBoostRegressor(**best_params, random_state=42)`. The selected model is re-trained on the *entire* training dataset (with target transformed by $\log 1p$) and predictions for submission are inverse-transformed and clipped to be non-negative.

2.5 CatBoost Regressor

Model overview. A CatBoost regression model (`catboost.CatBoostRegressor`) is trained to predict `Transport_Cost`. CatBoost natively handles categorical variables, permitting the pipeline to pass categorical column names directly to the model instead of performing one-hot encoding. The pipeline uses deterministic preprocessing and feature engineering, a $\log 1p$ transform of the target for training stability, and Optuna (TPE sampler) for hyperparameter tuning. The best model is re-trained on the full training dataset and used to generate the final submission.

Preprocessing pipeline (explicit steps and ordering). All preprocessing listed below is executed prior to training and the same saved statistics are reused at inference.

1. Training statistics and numeric missing-value imputation.

On training data compute medians for numeric features and use them to fill missing values:

- `Supplier_Reliability_median`
- `Equipment_Height_median`
- `Equipment_Width_median`
- `Equipment_Weight_median`
- `Equipment_Value_median`

Any NaNs in the computed medians are replaced with 0 as a safety fallback. At inference the stored medians (from `train_stats`) are reused.

2. Categorical missing-value imputation.

Compute modes (most frequent values) on training and fill missing values for: `Equipment_Type`, `Transport_Method`, `Rural_Hospital`. If a mode cannot be computed (empty), sensible defaults are used (`'Type A'`, `'Road'`, `'No'`).

3. Datetime parsing and date-derived features.

Parse `Order_Placed_Date` and `Delivery_Date` with `pd.to_datetime(..., errors='coerce')`. Derive:

- `Delivery_Time_Days` = days between delivery and order (negative or missing values clipped to 0).
- `Order_Year`, `Order_Month`, `Order_Day_of_Week`.

Fill any date-derived NaNs with the respective training-mode values (or safe defaults).

4. Physical / value feature engineering.

Construct features that capture size/value relationships:

- $\text{Value_per_Weight} = \frac{\text{Equipment_Value}}{\text{Equipment_Weight} + 1}$
- $\text{Equipment_Area} = \text{Equipment_Height} \times \text{Equipment_Width}$
- `Is_Weekend` = indicator for `Order_Day_of_Week` in `{5,6}`

5. **Binary mapping for boolean-like columns.**

Map the following to numeric flags (1/0) and fill missing with 0 if needed: `CrossBorder_Shipping`, `Urgent_Shipping`, `Installation_Service`, `Fragile_Equipment`, `Rural_Hospital`.

6. **Hospital_Info mapping.**

Map `Hospital_Info` to `{Wealthy:1, Working_Class:0}` and fill missing values with 0.

7. **No one-hot encoding for categorical features.**

Deliberate design choice: `Equipment_Type` and `Transport_Method` are left as categorical string columns. These column names are provided to CatBoost via the `cat_features` argument so CatBoost can apply its optimized categorical handling (ordered target statistics, hashing, etc.). This avoids the high-cardinality explosion and preserves native categorical semantics.

8. **Drop identifiers and raw date columns.**

Remove `Hospital_Id`, `Supplier_Name`, `Hospital_Location`, `Order_Placed_Date`, and `Delivery_Date` as non-predictive or leaking fields.

9. **Target handling.**

Clip `Transport_Cost` at 0 and apply $\log 1p$ before training: $\tilde{y} = \log(1 + y)$. During validation and final prediction invert via `np.exp`.

Why this preprocessing is suitable for CatBoost.

- CatBoost’s native categorical handling avoids expensive one-hot encodings and often produces better encodings for medium/high-cardinality categorical variables (via ordered statistics and built-in regularization).
- Imputation with medians/modes and safe defaults provides robust behavior for missing values and maintains stable numeric distributions.
- Derived date and physical features encode domain knowledge (lead time, size-to-value relationships) that CatBoost can exploit through its splitting and leaf-value estimations.
- Using $\log 1p$ on the target stabilizes training and reduces the influence of scarce extreme transport costs.

Hyperparameter tuning (procedure and settings).

- **Tuning framework:** Optuna with the TPE sampler (`optuna.samplers.TPESampler(seed=42)`), optimizing for minimum RMSE on a hold-out validation set.
- **Data split used for tuning:** A static 80/20 train/validation split is used (`train_test_split(..., test_size=0.2, random_state=42)`). The training portion is log-transformed; validation remains in original units for RMSE calculation after inverse-transform of predictions.
- **Categorical feature specification:** `categorical_features = ['Equipment_Type', 'Transport_Method']`. These names are passed to `CatBoostRegressor.fit(..., cat_features=categorical_features)` both during Optuna trials and final training.
- **Search space (as implemented):**
 - `iterations`: integer in `[100, 1000]`
 - `learning_rate`: float in `[0.01, 0.3]` (log sampling)
 - `depth`: integer in `[4, 10]`
 - `l2_leaf_reg`: float in `[1e-8, 100]` (log sampling)

- `random_seed`: fixed to 42 for reproducibility
- `verbose`: set to `False` during tuning to suppress training output
- **Trials:** `n_trials = 150` Optuna trials are run using the TPE sampler.
- **Metric:** Root Mean Squared Error (RMSE) computed in original cost units. Predictions from the log-target model are inversed with $\exp(\cdot) - 1$ before RMSE is computed against the validation target.
- **Final selection and retraining:** After Optuna returns `best_params`, the final `CatBoostRegressor(**best_params, random_seed=42, verbose=False)` is trained on the entire training data (with $\log 1p$ target) using the same `categorical_features` list. Predictions for submission are inverse-transformed and clipped to ≥ 0 .

Validation notes and behavior.

- Using a single static hold-out provides a fast estimate of generalization but is more sensitive to the particular split. If compute allows, replace the static split with k -fold CV (ensuring consistent `cat_features` handling across folds) for more robust hyperparameter assessment.
- CatBoost internally handles categorical encodings and ordering; when using CV or repeated experiments, keep the random seed fixed to ease reproducibility.
- Early stopping was not used in the provided tuning loop; if training time or overfitting becomes a concern, consider adding `early_stopping_rounds` and using a small eval set inside training.

2.6 Gradient Boosting Regressor

Model overview. A Gradient Boosting regressor (`sklearn.ensemble.GradientBoostingRegressor`) is trained to predict `Transport_Cost`. The pipeline uses deterministic, hand-crafted preprocessing and feature engineering, trains the model on a $\log 1p$ -transformed target to stabilise variance, evaluates performance on an 80/20 hold-out validation set (RMSE reported in original units), and finally re-trains on the full dataset to generate the submission file `submission_gb.csv`.

Preprocessing pipeline (explicit steps and ordering). All preprocessing below is applied before training and the same training statistics are reused at inference.

1. Compute and store training statistics for numeric imputation.

When `is_train=True` the code computes medians for the following numeric features and stores them in `train_stats`:

- `Supplier_Reliability_median`
- `Equipment_Height_median`
- `Equipment_Width_median`
- `Equipment_Weight_median`
- `Equipment_Value_median`

At inference these stored medians are used for imputation to ensure identical behaviour.

2. Numeric missing-value imputation.

Fill missing values in numeric columns `Supplier_Reliability`, `Equipment_Height`, `Equipment_Width`, `Equipment_Weight`, and `Equipment_Value` using the corresponding medians from `train_stats`.

3. Categorical missing-value imputation.

Compute the mode (most frequent value) on training and fill missing values for: `Equipment_Type`, `Transport_Method`, and `Rural_Hospital`. The computed modes are saved in `train_stats` and reused at inference.

4. Date parsing and date-derived features.

Parse `Order_Placed_Date` and `Delivery_Date` using `pd.to_datetime(..., errors='coerce')`. Derive:

- `Delivery_Time_Days` = number of days between delivery and order, with negative or missing values clipped to 0.
- `Order_Year`, `Order_Month`, `Order_Day_of_Week`.

Any derived date fields with missing values are filled using the corresponding training-mode values saved in `train_stats`.

5. Physical / value feature engineering.

Compute domain-informed numeric features:

- $\text{Value_per_Weight} = \frac{\text{Equipment_Value}}{\text{Equipment_Weight} + 1}$
- `Equipment_Area` = `Equipment_Height` × `Equipment_Width`
- `Is_Weekend` = indicator if `Order_Day_of_Week` is Saturday/Sunday

6. Binary mapping for boolean-like columns.

Convert yes/no fields to numeric flags (1/0). The code handles the presence/absence of the 'Yes' label robustly and fills missing with 0. Columns processed: `CrossBorder_Shipping`, `Urgent_Shipping`, `Installation_Service`, `Fragile_Equipment`, `Rural_Hospital`.

7. Hospital_Info mapping.

Map `Hospital_Info` to numeric values: {Wealthy:1, Working Class:0}; if `Wealthy` does not appear, everything is mapped to 0 and missing values are filled accordingly.

8. One-hot encoding for multi-class categoricals.

Apply `pd.get_dummies(..., drop_first=True)` to `Equipment_Type` and `Transport_Method` to create indicator variables while avoiding perfect multicollinearity.

9. Drop identifiers and raw date columns.

Remove non-predictive or potentially leaking columns: `Hospital_Id`, `Supplier_Name`, `Hospital_Location`, `Order_Placed_Date`, and `Delivery_Date`.

10. Target transform.

Clip `Transport_Cost` at 0, then compute $y_{\log} = \log 1p(\text{Transport_Cost})$. This transformed target is used for model training; predictions are inverse-transformed with $\exp(\cdot) - 1$ for evaluation and submission.

Model training and validation procedure.

- **Hold-out validation:** The code creates an 80/20 train/validation split using `train_test_split(..., test_size=0.2, random_state=42)`. The training portion is log-transformed ($\log 1p$) for stability.
- **Fixed hyperparameters:** The Gradient Boosting model is trained with the following hyperparameters:
 - `n_estimators` = 500

- `learning_rate = 0.05`
- `max_depth = 4`
- `subsample = 0.8`
- `loss = 'squared_error'`
- `random_state = 42`

- **Training:** The model is fit on the log-transformed training split.
- **Validation / metric:** Predictions on the validation set are inverse-transformed from log to original units using `np.exp1`. RMSE is computed in the original dollar units:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

where y_i are true transport costs and $\hat{y}_i$ are inverse-transformed predictions.

- **Reporting:** The script prints validation MSE and RMSE for the hold-out set.

Retraining and submission generation. After validating, the model is re-trained on 100% of the training data (using the same hyperparameters and the log $1p$ target). Final predictions on `test.csv` are inverse-transformed with `np.exp1`, clipped to be non-negative, and written to the submission file with columns `{Hospital_Id, Transport_Cost}`.

- **Fixed hyperparameters:** The current script uses a single hand-tuned hyperparameter set. For improved performance and robustness, consider hyperparameter optimization (Optuna, RandomizedSearchCV) over `n_estimators`, `learning_rate`, `max_depth`, `subsample`, and `min_samples_leaf`.
- **Cross-validation:** The single 80/20 hold-out gives a quick validation signal but can be noisy. Replacing it with k -fold CV (e.g., 5-fold) will produce more stable estimates of generalization performance.
- **Early stopping:** Although GradientBoostingRegressor from scikit-learn does not support native early stopping in the same way some libraries do, you can monitor validation error across staged predictions (`staged_predict`) to implement early stopping or switch to HistGradientBoostingRegressor or XGBoost/CatBoost/LightGBM for built-in early stopping.
- **Outlier handling:** Transport cost distributions often have heavy tails. Consider win-sorizing extreme targets or using robust losses (e.g., Huber) if outliers degrade validation stability.
- **Feature selection / regularization:** Try reducing correlated dummy columns, adding `min_samples_leaf`, or using regularised gradient boosting variants to avoid overfitting.

3 Comparisons of models on the basis of their respective kaggle score

Table 2: Kaggle Leaderboard Scores for Different Models

Model	Kaggle Score
XGBoost	38,767,56604.375
Random Forest	46,849,89841.144
CatBoost	48,497,82813.369
Gradient Boosting	53,357,18020.125
AdaBoost	56,681,69936.815
Linear regression	59,813,95947.328

From Table 2, it can be observed that the **XGBoost model** achieved the lowest Kaggle score of **38,767,56604.375**, thereby outperforming all other models evaluated in this study.

Reason for superior performance. The outstanding performance of XGBoost can be attributed to several factors:

- **Advanced regularization:** XGBoost introduces both L1 and L2 regularization terms, effectively controlling overfitting compared to traditional ensemble models like Random Forest or AdaBoost.
- **Optimized hyperparameters:** The model underwent extensive fine-tuning using Optuna with a 5-Fold Cross-Validation framework, ensuring an optimal balance between bias and variance.
- **Robustness to outliers and heterogeneous features:** The tree-based structure naturally accommodates both categorical and continuous variables, while the log-transformation of the target (`Transport_Cost`) further stabilized variance.
- **Efficient training with early stopping:** Early stopping prevented unnecessary iterations and ensured the model generalized well on unseen data, avoiding overfitting to the training set.

4 Hyperparameter tuning analysis and optimization history plot

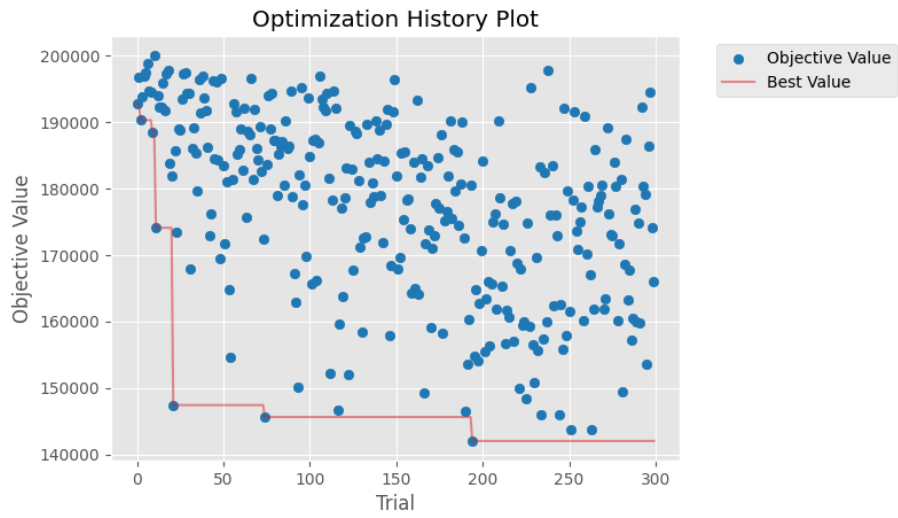


Figure 1: XgBoost (5 fold)

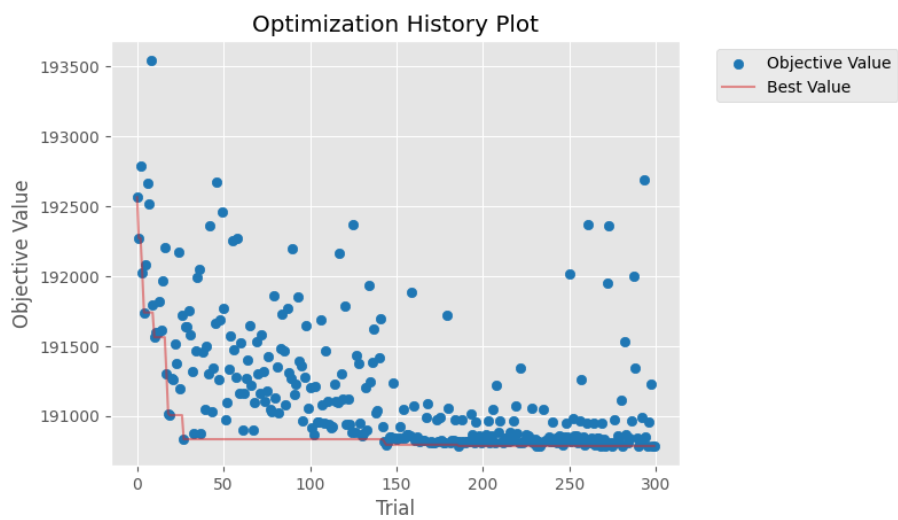


Figure 2: Random Forest (5 fold)

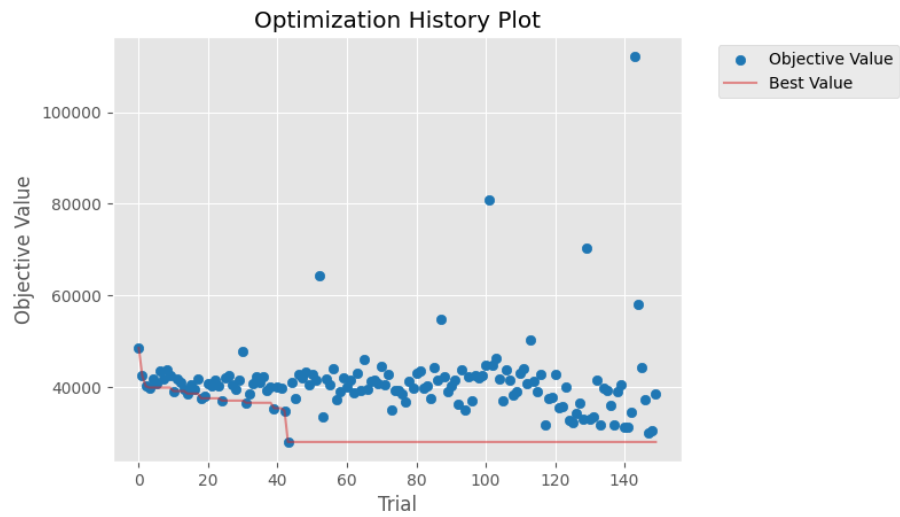


Figure 3: CatBoost

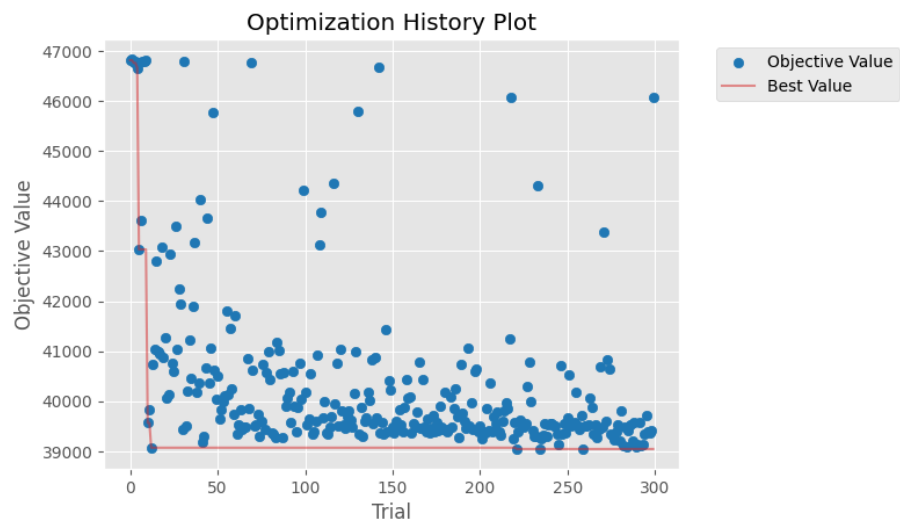


Figure 4: AdaBoost

5 GitHub repository link

Click [here](#) to navigate to the Github repository